

MSYNC

Prior predictive checks and simulation-based calibration

I. S. Plank

2025-08-06

Contents

1	Introduction	1
1.1	Some general settings	1
1.2	Package versions	1
1.3	General info	2
1.4	Load the data	2
1.5	Setting weakly informed priors	2
2	Conversation task	3
2.1	Level of the dyad	3
2.2	Level of the individual	7
2.3	Bernoulli model to predict order	11
3	Mirror game	14
3.1	Level of the dyad	14
3.2	Level of the individual	18

1 Introduction

This R Markdown script analyses data from the MSYNC project. The data was preprocessed before being read into this script. Here, we focus on the behavioural and the eye-tracking data.

1.1 Some general settings

```
# number of simulations
nsim = 250

# set number of iterations and warmup for models
iter = 3000
warm = 1000
```

1.2 Package versions

The following packages are used in this RMarkdown file:

```
## [1] "R version 4.5.1 (2025-06-13)"
## [1] "knitr version 1.50"
## [1] "ggplot2 version 3.5.2"
## [1] "brms version 2.22.0"
## [1] "designr version 0.1.13"
## [1] "bridgesampling version 1.1.2"
## [1] "tidyverse version 2.0.0"
## [1] "ggpubr version 0.6.1"
## [1] "gggrain version 0.0.4"
## [1] "bayesplot version 1.13.0"
```

```
## [1] "SBC version 0.3.0.9000"
## [1] "rstatix version 0.7.2"
## [1] "BayesFactor version 0.9.12.4.7"
## [1] "FactoMineR version 2.11"
## [1] "easystats version 0.7.5"
## [1] "psych version 2.5.6"
```

1.3 General info

In this RMarkdown file, we perform prior predictive checks as proposed in Schad, Betancourt and Vasishth (2020) using the SBC package. To do so, we create 250 simulated datasets where parameters are simulated from the priors. These parameters are used to create one fake dataset. Both the true underlying parameters and the simulated discrimination values are saved. Then, we create graphs showing the prior predictive distribution of the simulated discrimination threshold to check whether our priors fit our general expectations about the data. Next, we perform checks of computational faithfulness and model sensitivity as proposed by Schad, Betancourt and Vasishth (2020) and implemented in the SBC package. We create models for each of the simulated datasets. Last, we calculate performance metrics for each of these models, focusing on the population-level parameters.

We perform five SBCs: one for each task for data investigated on the level of the individual and one on the level of the dyad. These four SBCs are all based on the same priors and contain a random intercept, which is dropped in some of the actual models. Additionally, we perform one for a model with a Bernoulli distribution to assess the influence of PCs on order in the CT task.

1.4 Load the data

```
# check if the data file exists, if yes load it:
if (!file.exists("MSYNC_data.RData")) {

  # set the path to the data folders
  if (Sys.getenv("LOGNAME") == "vilya") {
    dt.path = "/Users/vilya/Documents/MSYNC/data"
  } else {
    dt.path = "/media/emba/emba-2/MSYNC/data"
  }

  createMSYNCdata(dt.path)

} else {

  load("MSYNC_data.RData")

}
```

1.5 Setting weakly informed priors

We will use the same priors for all of our Gaussian models, but different priors for the Bernoulli model.

```
priors = c(
  prior(normal(0, 0.10), class = Intercept), # because standardised
  prior(normal(1.0, 0.50), class = sigma),
  prior(normal(0.5, 0.50), class = sd), # less than sigma
  prior(normal(0, 0.30), class = b) # small to medium effect
)

kable(priors, caption = "Priors for all Gaussian linear (mixed) models")
```

Table 1: Priors for all Gaussian linear (mixed) models

prior	class	coef	group	resp	dpar	nlpar	lb	ub	source
normal(0, 0.1)	Intercept						NA	NA	user
normal(1, 0.5)	sigma						NA	NA	user
normal(0.5, 0.5)	sd						NA	NA	user
normal(0, 0.3)	b						NA	NA	user

```
# set weakly informative priors
priors.b = c(
  prior(normal(0, 0.50), class = Intercept),
  prior(normal(0, 1.00), class = b)
)

kable(priors.b, caption = "Priors for the Bernoulli linear model")
```

Table 2: Priors for the Bernoulli linear model

prior	class	coef	group	resp	dpar	nlpar	lb	ub	source
normal(0, 0.5)	Intercept						NA	NA	user
normal(0, 1)	b						NA	NA	user

2 Conversation task

2.1 Level of the dyad

```
# set the contrast
contrasts(df.dyad.ct.mea$order) = contr.sum(2)
contrasts(df.dyad.ct.mea$order)

##      [,1]
## CT-MG    1
## MG-CT   -1

# code for filenames
code = "CT_order"

# scale the variables
df.dyad.ct.mea = df.dyad.ct.mea %>%
  mutate(across(where(is.numeric),
    .fns = scale_this,
    .names = "s{col}" ) )

# model formula
f = brms::bf( sMEA.mean ~ order + sMEA.total.mov)
f

## sMEA.mean ~ order + sMEA.total.mov

# check if the SBC already exists
if (file.exists(file.path(cache_dir, sprintf("df_res_%s.rds", code)))) {
  # load in the results of the SBC
  df.results = readRDS(file.path(cache_dir, sprintf("df_res_%s.rds", code)))
  df.backend = readRDS(file.path(cache_dir, sprintf("df_div_%s.rds", code)))
  dat        = readRDS(file.path(cache_dir, sprintf("dat_%s.rds", code)))
} else {
  # stimulate some data
  set.seed(2244)
  gen = SBC_generator_brms(f, data = df.dyad.ct.mea,
```

```

        prior = priors %>% filter(class != "sd"),
    thin = 50, warmup = 10000, refresh = 2000,
    generate_lp = TRUE, init = 0.1)
bck = SBC_backend_brms_from_generator(gen, chains = 4, thin = 1,
    warmup = warm, iter = iter)
dat = generate_datasets(gen, nsim)
saveRDS(dat, file.path(cache_dir, sprintf("dat_%s.rds", code)))
# perform the SBC
res = compute_SBC(
    dat,
    bck,
    cache_mode = "results",
    cache_location = file.path(cache_dir, sprintf("res_%s", code)))
df.results = res$stats
df.backend = res$backend_diagnostics
saveRDS(df.results, file = file.path(cache_dir, paste0("df_res_", code, ".rds")))
saveRDS(df.backend,
    file = file.path(cache_dir, paste0("df_div_", code, ".rds")))
}

```

We start by investigating the rhats and the number of divergent samples. This shows that only 0 of 250 simulations had at least one parameter that had an rhat of at least 1.05 and 0 models had divergent samples. This suggests that this model performs well.

Next, we can plot the simulated values to perform prior predictive checks.

```

dvfakemat = matrix(NA, nrow(dat[['generated']][[1]]), length(dat[['generated']]))
for (i in 1:length(dat[['generated']])) {
    dvfakemat[,i] = dat[['generated']][[i]][[1]]
}

# compute one histogram per simulated data-set
dvfakematH = dvfakemat
dvfakematH[dvfakematH > 4] = 4
dvfakematH[dvfakematH < -4] = -4
breaks = seq(min(dvfakematH, na.rm=T), max(dvfakematH, na.rm=T), length.out = 101)
binwidth = breaks[2] - breaks[1]
histmat = matrix(NA, ncol = dim(dvfakemat)[2], nrow = length(breaks)-1)
for (i in 1:dim(dvfakemat)[2]) {
    histmat[,i] = hist(dvfakematH[,i], breaks = breaks, plot = F)$counts
}

# for each bin, compute quantiles across histograms
probs = seq(0.1, 0.9, 0.1)
quantmat = as.data.frame(matrix(NA, nrow=dim(histmat)[1], ncol = length(probs)))
names(quantmat) = paste0("p", probs)
for (i in 1:dim(histmat)[1]) {
    quantmat[i,] = quantile(histmat[i,], p = probs)
}

quantmat$x = breaks[2:length(breaks)] - binwidth/2 # add bin mean
p1 = ggplot(data = quantmat, aes(x = x)) +
    geom_ribbon(aes(ymax = p0.9, ymin = p0.1), fill = c_light) +
    geom_ribbon(aes(ymax = p0.8, ymin = p0.2), fill = c_light_highlight) +
    geom_ribbon(aes(ymax = p0.7, ymin = p0.3), fill = c_mid) +
    geom_ribbon(aes(ymax = p0.6, ymin = p0.4), fill = c_mid_highlight) +
    geom_line(aes(y = p0.5), colour = c_dark, linewidth = 1) +
    labs(title = "Distribution of simulated betas", y = "", x = "") +
    theme_bw()

tmpM = apply(dvfakemat, 2, mean) # mean
tmpSD = apply(dvfakemat, 2, sd)
p2 = ggplot() +

```

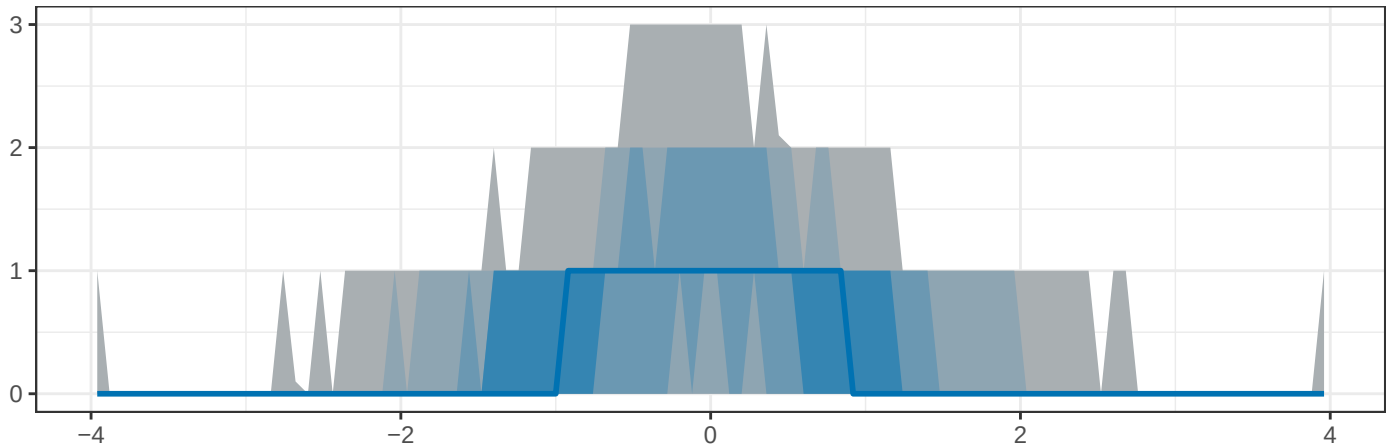
```

stat_bin(aes(x = tmpM), fill = c_dark) +
labs(x = "Mean beta", title = "Means of simulated betas") +
theme_bw()
p3 = ggplot() +
stat_bin(aes(x = tmpSD), fill = c_dark) +
labs(x = "SD beta", title = "SDs of simulated betas") +
theme_bw()
p = ggarrange(p1,
ggarrange(p2, p3, ncol = 2, labels = c("B", "C")),
nrow = 2, labels = "A")
annotate_figure(p, top = text_grob("Prior predictive checks", face = "bold", size = 14))

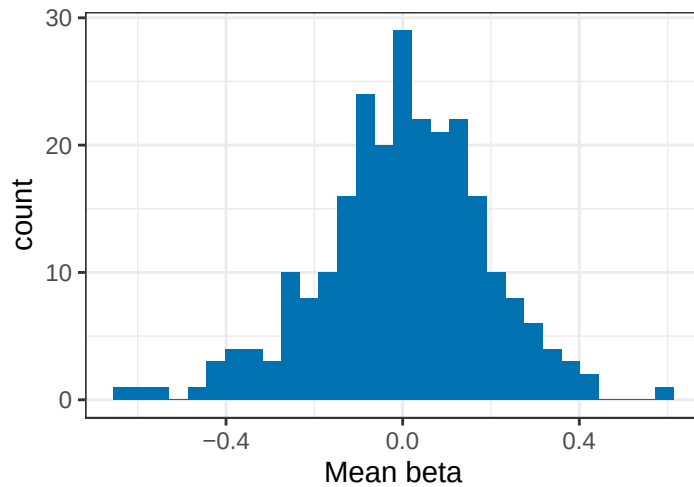
```

Prior predictive checks

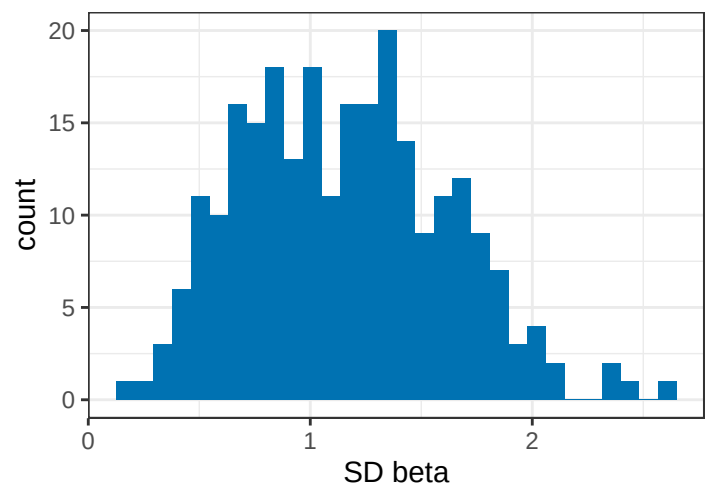
A Distribution of simulated betas



B Means of simulated betas



C SDs of simulated betas



This all looks good and we go ahead with these priors and check the results of the SBC. We only plot the results from the models that had no divergence issues.

```

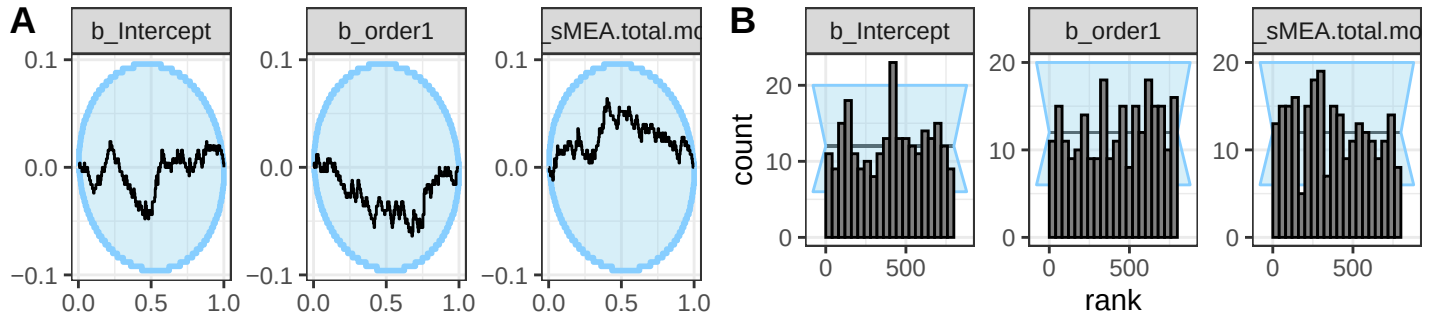
# get simulation numbers with issues
mx_rnk = max(df.results$max_rank)
check = merge(df.results %>%
  group_by(sim_id) %>%
  summarise(rhat = max(rhat, na.rm = T),
            max_rank = mean(max_rank)) %>%
  filter(rhat >= 1.05 | max_rank != mx_rnk),
df.backend %>% filter(n_divergent > 0), all = T)

```

```
# plot SBC with functions from the SBC package focusing on population-level parameters
df.results.b = df.results %>%
  filter(substr(variable, 1, 2) == "b_") %>%
  filter(!(sim_id %in% check$sim_id))
p1 = plot_ecdf_diff(df.results.b) + theme_bw() + theme(legend.position = "none") +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p2 = plot_rank_hist(df.results.b, bins = 20) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))

p = ggarrange(p1, p2, labels = "A", ncol = 2, nrow = 1)
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
  face = "bold", size = 14))
```

Computational faithfulness and model sensitivity

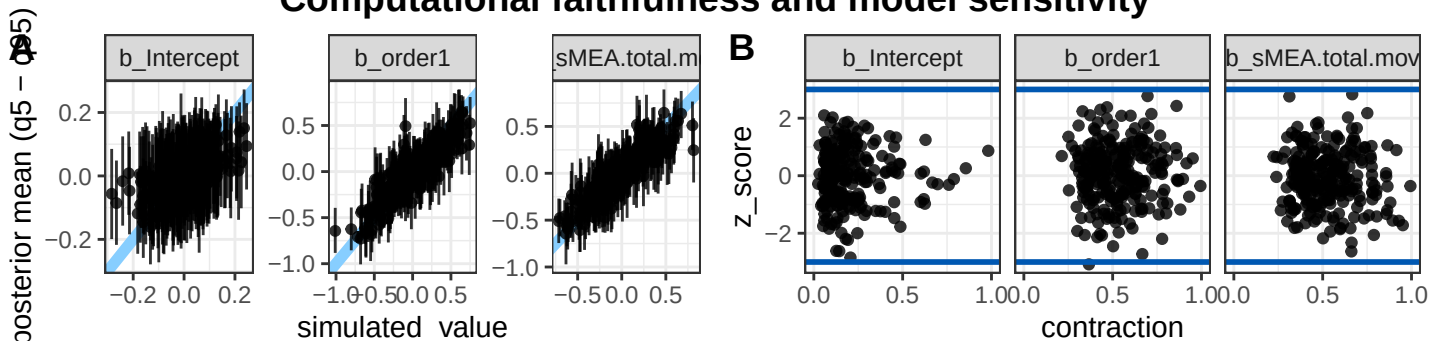


All looks good and we judge this to be acceptable as we cannot identify clear bias patterns as described in the information accompanying the SBC package: https://hyunjimoon.github.io/SBC/articles/rank_visualizations.html

```
p3 = plot_sim_estimated(df.results.b, alpha = .8) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p4 = plot_contraction(df.results.b,
  prior_sd =
    setNames(c(0.1, rep(0.3, times = length(unique(df.results.b$variable))-1)),
      unique(df.results.b$variable))) +
  theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3)) +
  geom_hline(yintercept = c(-3, 3), colour = c_dark_highlight, linewidth = 1)

p = ggarrange(p3, p4, labels = "B", ncol = 2, nrow = 1)
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
  face = "bold", size = 14))
```

Computational faithfulness and model sensitivity



Then, we investigated the relationship between the simulated true parameters and the posterior estimates. Most parameters were recovered successfully within an uncertainty interval of $\alpha = 0.05$. Last, z-score and the posterior contraction of our population-level predictors look good as well.

2.2 Level of the individual

```
# scale the relevant data
df.indi.ct.pc = merge(df.indi.ct.mea,
                      df.indi %>% select(subID, RAADS_final)) %>%

  mutate(
    sRAADS.final = scale_this(RAADS_final),
    sMEA.mean = scale_this(MEA.mean),
    sMEA.total.mov = scale_this(MEA.total.mov)
  )

# set the contrast
contrasts(df.indi.ct.pc$order) = contr.sum(2)
contrasts(df.indi.ct.pc$order)

##          [,1]
## CT-MG      1
## MG-CT     -1

# code for filenames
code = "CT_order_raads"

# model formula
f = brms::bf( sMEA.mean ~ sRAADS.final * order + sMEA.total.mov + (1 | dyad))
f

## sMEA.mean ~ sRAADS.final * order + sMEA.total.mov + (1 | dyad)

# check if the SBC already exists
if (file.exists(file.path(cache_dir, sprintf("df_res_%s.rds", code)))) {
  # load in the results of the SBC
  df.results = readRDS(file.path(cache_dir, sprintf("df_res_%s.rds", code)))
  df.backend = readRDS(file.path(cache_dir, sprintf("df_div_%s.rds", code)))
  dat = readRDS(file.path(cache_dir, sprintf("dat_%s.rds", code)))
} else {
  # stimulate some data
  set.seed(2486)
  gen = SBC_generator_brms(f, data = df.indi.ct.pc,
                           prior = priors,
                           thin = 50, warmup = 10000, refresh = 2000,
                           generate_lp = TRUE, init = 0.1)
  bck = SBC_backend_brms_from_generator(gen, chains = 4, thin = 1,
                                         warmup = warm, iter = iter)
  dat = generate_datasets(gen, nsim)
  saveRDS(dat, file.path(cache_dir, sprintf("dat_%s.rds", code)))
  # perform the SBC
  res = compute_SBC(
    dat,
    bck,
    cache_mode = "results",
    cache_location = file.path(cache_dir, sprintf("res_%s", code)))
  df.results = res$stats
  df.backend = res$backend_diagnostics
  saveRDS(df.results, file = file.path(cache_dir, paste0("df_res_", code, ".rds")))
  saveRDS(df.backend,
          file = file.path(cache_dir, paste0("df_div_", code, ".rds")))
}
```

We start by investigating the rhats and the number of divergent samples. This shows that only 0 of 250 simulations had at least one parameter that had an rhat of at least 1.05, but 60 models had divergent samples. We will have to look out for those in the final model and if the problems persist possibly drop the random intercept.

Next, we can plot the simulated values to perform prior predictive checks.

```

dvfakemat = matrix(NA, nrow(dat[['generated']][[1]]), length(dat[['generated']]))
for (i in 1:length(dat[['generated']])) {
  dvfakemat[,i] = dat[['generated']][[i]][[1]]
}

# compute one histogram per simulated data-set
dvfakematH = dvfakemat
dvfakematH[dvfakematH > 4] = 4
dvfakematH[dvfakematH < -4] = -4
breaks = seq(min(dvfakematH, na.rm=T), max(dvfakematH, na.rm=T), length.out = 101)
binwidth = breaks[2] - breaks[1]
histmat = matrix(NA, ncol = dim(dvfakemat)[2], nrow = length(breaks)-1)
for (i in 1:dim(dvfakemat)[2]) {
  histmat[,i] = hist(dvfakematH[,i], breaks = breaks, plot = F)$counts
}

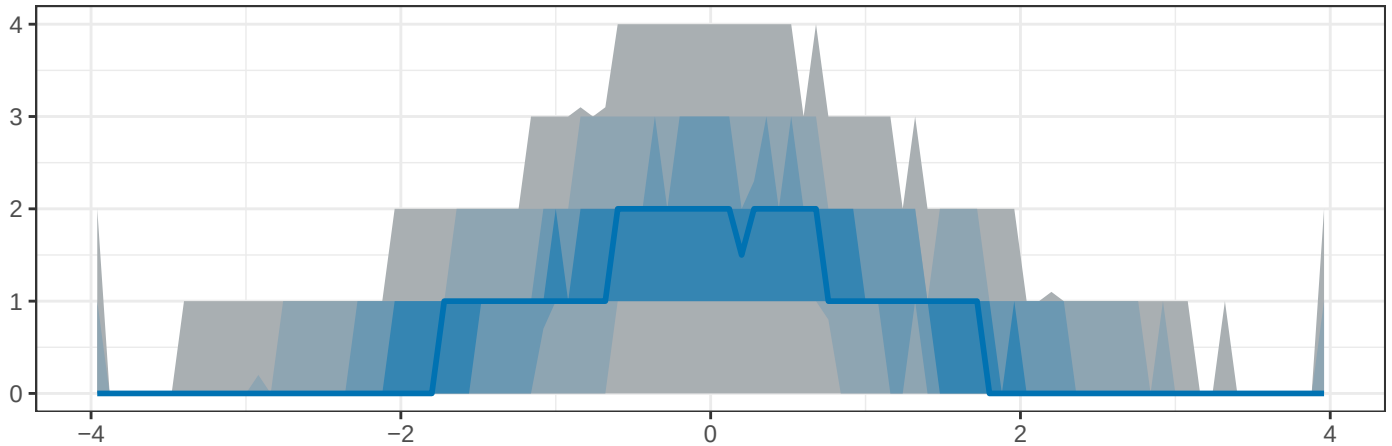
# for each bin, compute quantiles across histograms
probs = seq(0.1, 0.9, 0.1)
quantmat = as.data.frame(matrix(NA, nrow=dim(histmat)[1], ncol = length(probs)))
names(quantmat) = paste0("p", probs)
for (i in 1:dim(histmat)[1]) {
  quantmat[i,] = quantile(histmat[i,], p = probs)
}
quantmat$x = breaks[2:length(breaks)] - binwidth/2 # add bin mean
p1 = ggplot(data = quantmat, aes(x = x)) +
  geom_ribbon(aes(ymax = p0.9, ymin = p0.1), fill = c_light) +
  geom_ribbon(aes(ymax = p0.8, ymin = p0.2), fill = c_light_highlight) +
  geom_ribbon(aes(ymax = p0.7, ymin = p0.3), fill = c_mid) +
  geom_ribbon(aes(ymax = p0.6, ymin = p0.4), fill = c_mid_highlight) +
  geom_line(aes(y = p0.5), colour = c_dark, linewidth = 1) +
  labs(title = "Distribution of simulated betas", y = "", x = "") +
  theme_bw()

tmpM = apply(dvfakemat, 2, mean) # mean
tmpSD = apply(dvfakemat, 2, sd)
p2 = ggplot() +
  stat_bin(aes(x = tmpM), fill = c_dark) +
  labs(x = "Mean beta", title = "Means of simulated betas") +
  theme_bw()
p3 = ggplot() +
  stat_bin(aes(x = tmpSD), fill = c_dark) +
  labs(x = "SD beta", title = "SDs of simulated betas") +
  theme_bw()
p = ggarrange(p1,
  ggarrange(p2, p3, ncol = 2, labels = c("B", "C")),
  nrow = 2, labels = "A")
annotate_figure(p, top = text_grob("Prior predictive checks", face = "bold", size = 14))

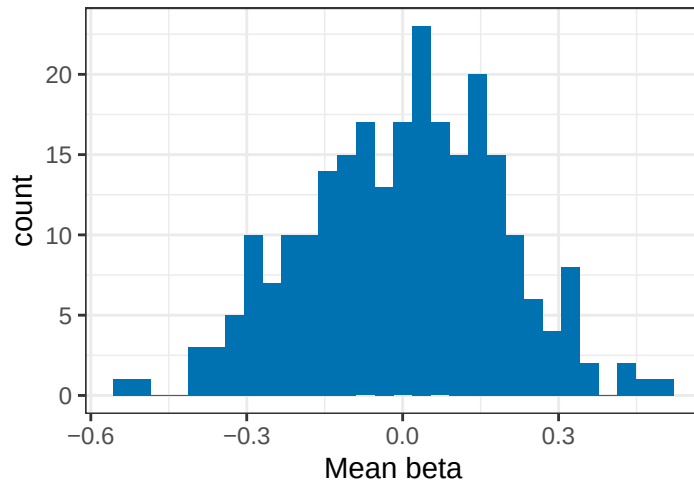
```

Prior predictive checks

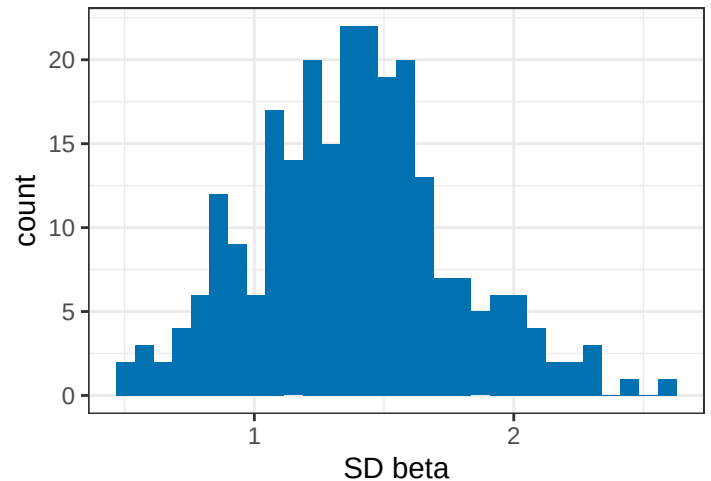
A Distribution of simulated betas



B Means of simulated betas



C SDs of simulated betas



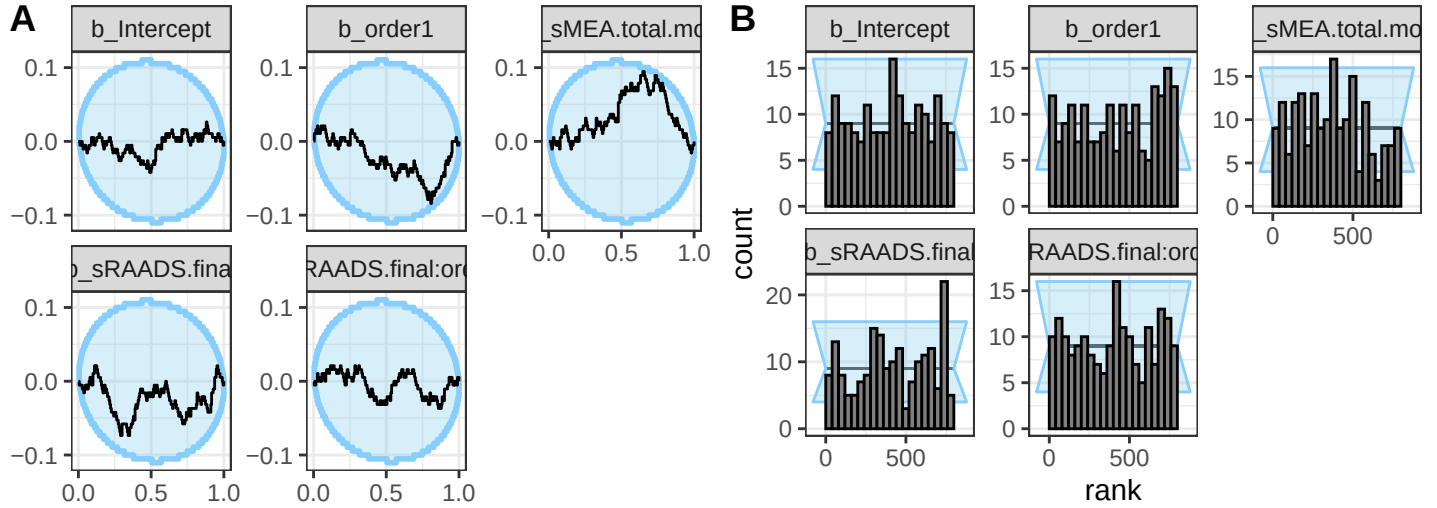
This all looks good and we go ahead with these priors and check the results of the SBC. We only plot the results from the models that had no divergence issues.

```
# get simulation numbers with issues
mx_rnk = max(df.results$max_rank)
check = merge(df.results %>%
  group_by(sim_id) %>%
  summarise(rhat = max(rhat, na.rm = T),
            max_rank = mean(max_rank)) %>%
  filter(rhat >= 1.05 | max_rank != mx_rnk),
  df.backend %>% filter(n_divergent > 0), all = T)

# plot SBC with functions from the SBC package focusing on population-level parameters
df.results.b = df.results %>%
  filter(substr(variable, 1, 2) == "b_") %>%
  filter(!(sim_id %in% check$sim_id))
p1 = plot_ecdf_diff(df.results.b) + theme_bw() + theme(legend.position = "none") +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p2 = plot_rank_hist(df.results.b, bins = 20) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p = ggarrange(p1, p2, labels = "AUTO", ncol = 2, nrow = 1)
```

```
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
                                   face = "bold", size = 14))
```

Computational faithfulness and model sensitivity

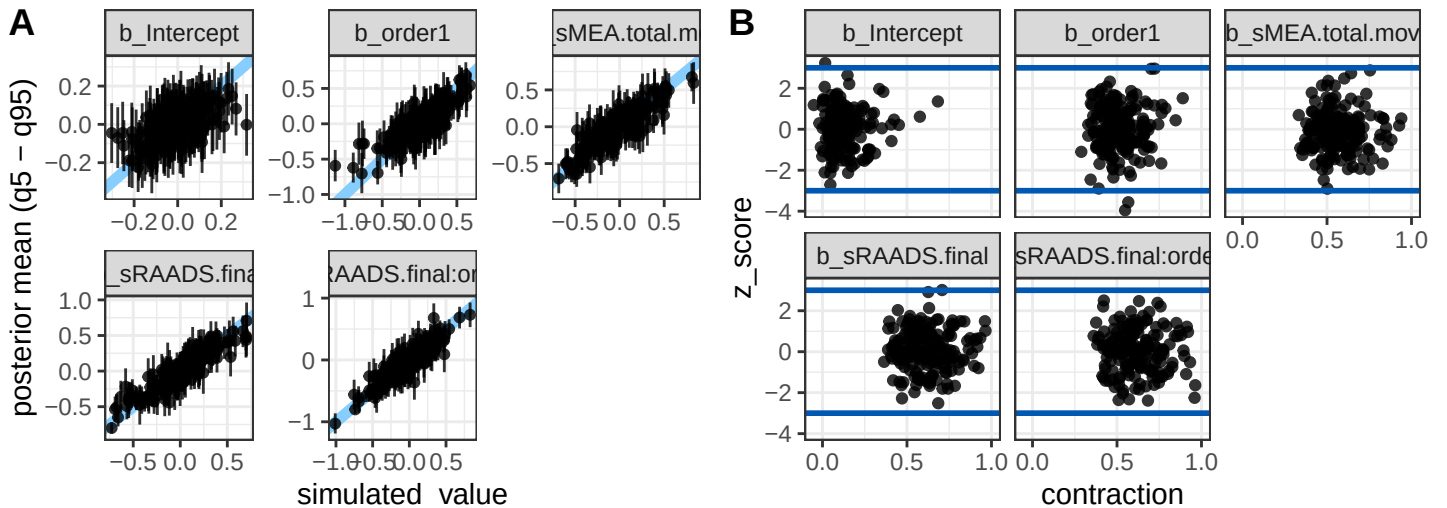


All looks good and we judge this to be acceptable as we cannot identify clear bias patterns as described in the information accompanying the SBC package: https://hyunjimoon.github.io/SBC/articles/rank_visualizations.html

```
p3 = plot_sim_estimated(df.results.b, alpha = .8) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p4 = plot_contraction(df.results.b,
  prior_sd =
    setNames(c(0.1, rep(0.3, times = length(unique(df.results.b$variable))-1)),
      unique(df.results.b$variable))) +
  theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3)) +
  geom_hline(yintercept = c(-3, 3), colour = c_dark_highlight, linewidth = 1)

p = ggarrange(p3, p4, labels = "AUTO", ncol = 2, nrow = 1)
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
                                   face = "bold", size = 14))
```

Computational faithfulness and model sensitivity



Then, we investigated the relationship between the simulated true parameters and the posterior estimates. Most parameters were recovered successfully within an uncertainty interval of $\alpha = 0.05$. Last, z-score and the posterior contraction of our population-level predictors look good as well.

2.3 Bernoulli model to predict order

```
# simulate scaled PCs
set.seed(4682)
df.indi.ct.mea$spc1 = rnorm(nrow(df.indi.ct.mea))
df.indi.ct.mea$spc2 = rnorm(nrow(df.indi.ct.mea))
df.indi.ct.mea$spc3 = rnorm(nrow(df.indi.ct.mea))
df.indi.ct.mea$spc4 = rnorm(nrow(df.indi.ct.mea))
df.indi.ct.mea = df.indi.ct.mea %>%
  mutate(
    order.bin = if_else(order == "MG-CT", 1, 0)
  )

code = "CT_pcs"

# model formula
f = brms::bf( order.bin ~ spc1 + spc2 + spc3 + spc4 )
f

## order.bin ~ spc1 + spc2 + spc3 + spc4
code = "CT_pc"

# check if the SBC already exists
if (file.exists(file.path(cache_dir, sprintf("df_res_%s.rds", code)))) {
  # load in the results of the SBC
  df.results = readRDS(file.path(cache_dir, sprintf("df_res_%s.rds", code)))
  df.backend = readRDS(file.path(cache_dir, sprintf("df_div_%s.rds", code)))
  dat = readRDS(file.path(cache_dir, sprintf("dat_%s.rds", code)))
} else {
  # stimulate some data
  set.seed(2246)
  gen = SBC_generator_brms(f, data = df.indi.ct.mea, prior = priors.b,
    thin = 50, warmup = 10000, refresh = 2000, family = bernoulli(link = "logit"),
    generate_lp = TRUE, init = 0.1)
  bck = SBC_backend_brms_from_generator(gen, chains = 4, thin = 1,
    warmup = warm, iter = iter)
  dat = generate_datasets(gen, nsim)
  saveRDS(dat, file.path(cache_dir, sprintf("dat_%s.rds", code)))
  # perform the SBC
  res = compute_SBC(
    dat,
    bck,
    cache_mode = "results",
    cache_location = file.path(cache_dir, sprintf("res_%s", code)))
  df.results = res$stats
  df.backend = res$backend_diagnostics
  saveRDS(df.results, file = file.path(cache_dir, paste0("df_res_", code, ".rds")))
  saveRDS(df.backend,
    file = file.path(cache_dir, paste0("df_div_", code, ".rds")))
}
```

We start by investigating the rhats and the number of divergent samples. This shows that 0 of 250 simulations had at least one parameter that had an rhat of at least 1.05 and 0 models had divergent samples. This suggests that this model performs well.

Next, we can plot the simulated values to perform prior predictive checks.

```
# create a matrix out of generated data
dvfakemat = matrix(NA, nrow(dat[['generated']][[1]]), length(dat[['generated']]))
for (i in 1:length(dat[['generated']])) {
  dvfakemat[,i] = dat[['generated']][[i]][[1]]
}
```

```

truePars = dat$variables

# compute one histogram per simulated data-set
options = c(0, 1)
histmat = matrix(NA, ncol = nrow(truePars), length(options))
for (i in 1:nrow(truePars)) {
  for (j in 1:length(options))
  {
    histmat[j,i] = sum(dvfakemat[,i] == options[j])
  }
}

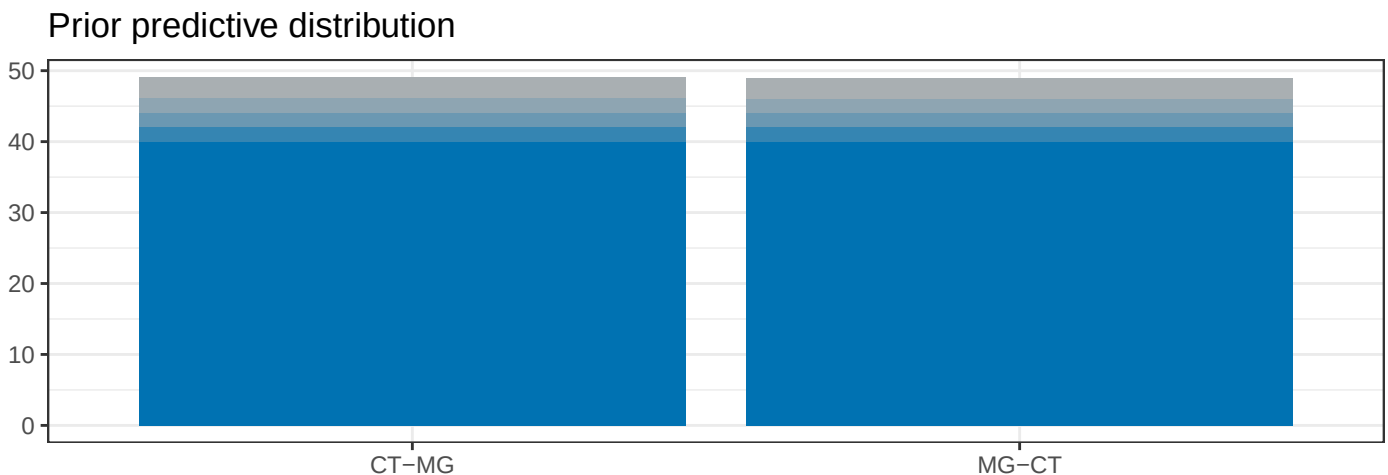
# for each bin, compute quantiles across histograms
probs = seq(0.1, 0.9, 0.1)
quantmat = as.data.frame(matrix(NA, nrow=dim(histmat)[1], ncol = length(probs)))
names(quantmat) = paste0("p", probs)
for (i in 1:dim(histmat)[1]) {
  quantmat[i,] = quantile(histmat[i,], p = probs)
}

quantmat$x = c("CT-MG", "MG-CT")
p0 = ggplot(data = quantmat, aes(x = x)) +
  geom_bar(aes(y = p0.9), fill = c_light, stat = "identity") +
  geom_bar(aes(y = p0.8), fill = c_light_highlight, stat = "identity") +
  geom_bar(aes(y = p0.7), fill = c_mid, stat = "identity") +
  geom_bar(aes(y = p0.6), fill = c_mid_highlight, stat = "identity") +
  geom_bar(aes(y = p0.5), fill = c_dark, stat = "identity") +
  labs(title = "Prior predictive distribution", y = "", x = "") +
  theme_bw()

annotate_figure(p0, top = text_grob("Prior predictive checks", face = "bold", size = 14))

```

Prior predictive checks



This all looks good and we go ahead with these priors and check the results of the SBC. We only plot the results from the models that had no divergence issues.

```

# get simulation numbers with issues
mx_rnk = max(df.results$max_rank)
check = merge(df.results %>%
  group_by(sim_id) %>%
  summarise(rhat = max(rhat, na.rm = T),
            max_rank = mean(max_rank)) %>%
  filter(rhat >= 1.05 | max_rank != mx_rnk),
  df.backend %>% filter(n_divergent > 0), all = T)

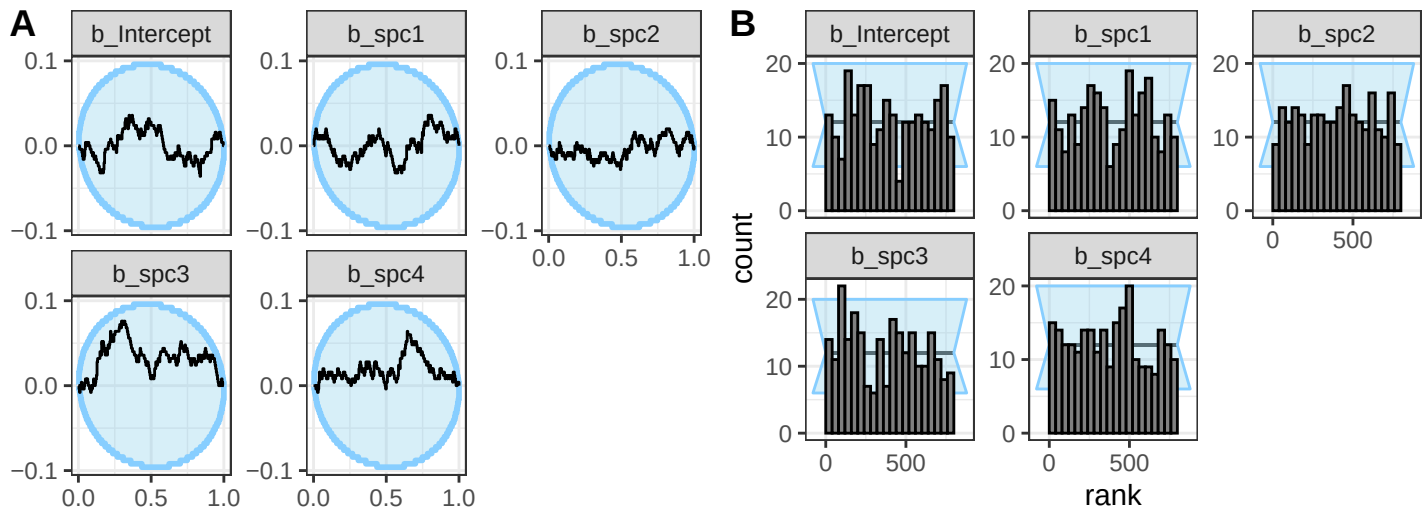
# plot SBC with functions from the SBC package focusing on population-level parameters

```

```
df.results.b = df.results %>%
  filter(substr(variable, 1, 2) == "b_") %>%
  filter(!(sim_id %in% check$sim_id))
p1 = plot_ecdf_diff(df.results.b) + theme_bw() + theme(legend.position = "none") +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p2 = plot_rank_hist(df.results.b, bins = 20) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))

p = ggarrange(p1, p2, labels = "A", ncol = 2, nrow = 1)
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
  face = "bold", size = 14))
```

Computational faithfulness and model sensitivity

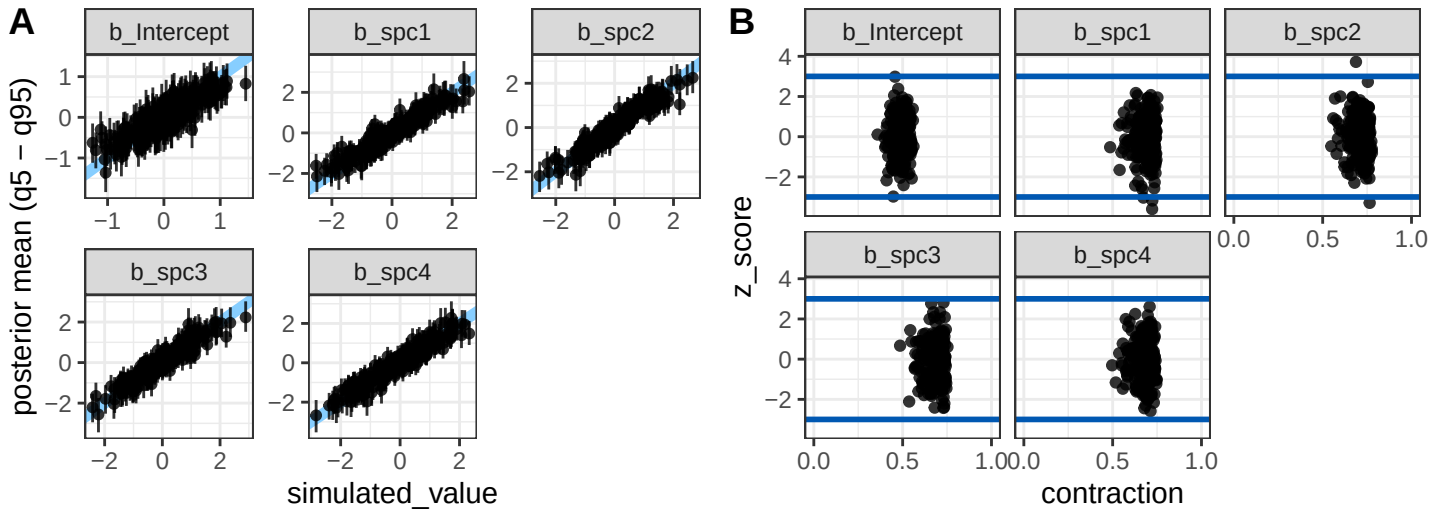


All looks good and we judge this to be acceptable as we cannot identify clear bias patterns as described in the information accompanying the SBC package: https://hyunjimoon.github.io/SBC/articles/rank_visualizations.html

```
p3 = plot_sim_estimated(df.results.b, alpha = .8) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p4 = plot_contraction(df.results.b,
  prior_sd =
    setNames(c(0.5, rep(1, times = length(unique(df.results.b$variable))-1)),
      unique(df.results.b$variable))) +
  theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3)) +
  geom_hline(yintercept = c(-3, 3), colour = c_dark_highlight, linewidth = 1)

p = ggarrange(p3, p4, labels = "A", ncol = 2, nrow = 1)
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
  face = "bold", size = 14))
```

Computational faithfulness and model sensitivity



Then, we investigated the relationship between the simulated true parameters and the posterior estimates. Most parameters were recovered successfully within an uncertainty interval of $\alpha = 0.05$. Last, z-score and the posterior contraction of our population-level predictors look good as well.

3 Mirror game

Since the mirror game has three phases, these are additionally added as covariates. We again model dyad as a group-level intercept. However, we do not add slopes for the phases, since it led to divergence issues.

3.1 Level of the dyad

```
# prepare the data frame
df.dyad.mg.mea = merge(df.dyad.mg.mea,
                        df.dyad %>% select(dyad, starts_with("RAADS"), starts_with("IRI"))) %>%
  mutate(across(where(is.numeric),
                 .fns = scale_this,
                 .names = "s{col}" ) )

# set the contrasts
contrasts(df.dyad.mg.mea$phase) = contr.sum(3)
contrasts(df.dyad.mg.mea$phase)

##      [,1] [,2]
## p1      1    0
## p2      0    1
## p3     -1   -1

contrasts(df.dyad.mg.mea$order) = contr.sum(2)
contrasts(df.dyad.mg.mea$order)

##      [,1]
## CT-MG      1
## MG-CT     -1

# code for filenames
code = "MG_dyad_mea_mean"

# model formula
f = brms::bf( sMEA.mean ~ sRAADS.diff + sIRI.diff + phase + order +
              (1 | dyad))
f

## sMEA.mean ~ sRAADS.diff + sIRI.diff + phase + order + (1 | dyad)
```

```

# check if the SBC already exists
if (file.exists(file.path(cache_dir, sprintf("df_res_%s.rds", code)))) {
  # load in the results of the SBC
  df.results = readRDS(file.path(cache_dir, sprintf("df_res_%s.rds", code)))
  df.backend = readRDS(file.path(cache_dir, sprintf("df_div_%s.rds", code)))
  dat = readRDS(file.path(cache_dir, sprintf("dat_%s.rds", code)))
} else {
  # stimulate some data
  set.seed(2846)
  gen = SBC_generator_brms(f, data = df.dyad.mg.mea,
    prior = priors %>% filter(class != "sd"),
    thin = 50, warmup = 10000, refresh = 2000,
    generate_lp = TRUE, init = 0.1)
  bck = SBC_backend_brms_from_generator(gen, chains = 4, thin = 1,
    warmup = warm, iter = iter)
  dat = generate_datasets(gen, nsim)
  saveRDS(dat, file.path(cache_dir, sprintf("dat_%s.rds", code)))
  # perform the SBC
  res = compute_SBC(
    dat,
    bck,
    cache_mode = "results",
    cache_location = file.path(cache_dir, sprintf("res_%s", code)))
  df.results = res$stats
  df.backend = res$backend_diagnostics
  saveRDS(df.results, file = file.path(cache_dir, paste0("df_res_", code, ".rds")))
  saveRDS(df.backend,
    file = file.path(cache_dir, paste0("df_div_", code, ".rds")))
}

```

We start by investigating the rhats and the number of divergent samples. This shows that only 0 of 250 simulations had at least one parameter that had an rhat of at least 1.05 and 14 models had divergent samples. This suggests that this model performs well, but we will check for divergent transitions in the final model.

Next, we can plot the simulated values to perform prior predictive checks.

```

dvfakemat = matrix(NA, nrow(dat[['generated']][[1]]), length(dat[['generated']]))
for (i in 1:length(dat[['generated']])) {
  dvfakemat[,i] = dat[['generated']][[i]][[1]]
}

# compute one histogram per simulated data-set
dvfakematH = dvfakemat
dvfakematH[dvfakematH > 4] = 4
dvfakematH[dvfakematH < -4] = -4
breaks = seq(min(dvfakematH, na.rm=T), max(dvfakematH, na.rm=T), length.out = 101)
binwidth = breaks[2] - breaks[1]
histmat = matrix(NA, ncol = dim(dvfakemat)[2], nrow = length(breaks)-1)
for (i in 1:dim(dvfakemat)[2]) {
  histmat[,i] = hist(dvfakematH[,i], breaks = breaks, plot = F)$counts
}

# for each bin, compute quantiles across histograms
probs = seq(0.1, 0.9, 0.1)
quantmat = as.data.frame(matrix(NA, nrow=dim(histmat)[1], ncol = length(probs)))
names(quantmat) = paste0("p", probs)
for (i in 1:dim(histmat)[1]) {
  quantmat[i,] = quantile(histmat[i,], p = probs)
}
quantmat$x = breaks[2:length(breaks)] - binwidth/2 # add bin mean
p1 = ggplot(data = quantmat, aes(x = x)) +
  geom_ribbon(aes(ymax = p0.9, ymin = p0.1), fill = c_light) +

```

```

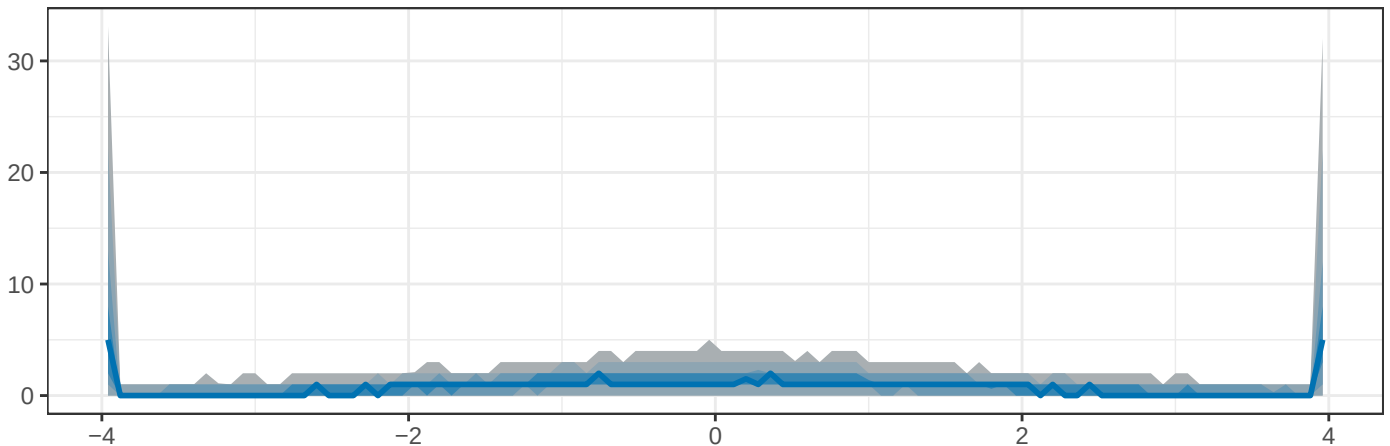
geom_ribbon(aes(ymax = p0.8, ymin = p0.2), fill = c_light_highlight) +
geom_ribbon(aes(ymax = p0.7, ymin = p0.3), fill = c_mid) +
geom_ribbon(aes(ymax = p0.6, ymin = p0.4), fill = c_mid_highlight) +
geom_line(aes(y = p0.5), colour = c_dark, linewidth = 1) +
labs(title = "Distribution of simulated betas", y = "", x = "") +
theme_bw()

tmpM = apply(dvfakemat, 2, mean) # mean
tmpSD = apply(dvfakemat, 2, sd)
p2 = ggplot() +
  stat_bin(aes(x = tmpM), fill = c_dark) +
  labs(x = "Mean beta", title = "Means of simulated betas") +
  theme_bw()
p3 = ggplot() +
  stat_bin(aes(x = tmpSD), fill = c_dark) +
  labs(x = "SD beta", title = "SDs of simulated betas") +
  theme_bw()
p = ggarrange(p1,
  ggarrange(p2, p3, ncol = 2, labels = c("B", "C")),
  nrow = 2, labels = "A")
annotate_figure(p, top = text_grob("Prior predictive checks", face = "bold", size = 14))

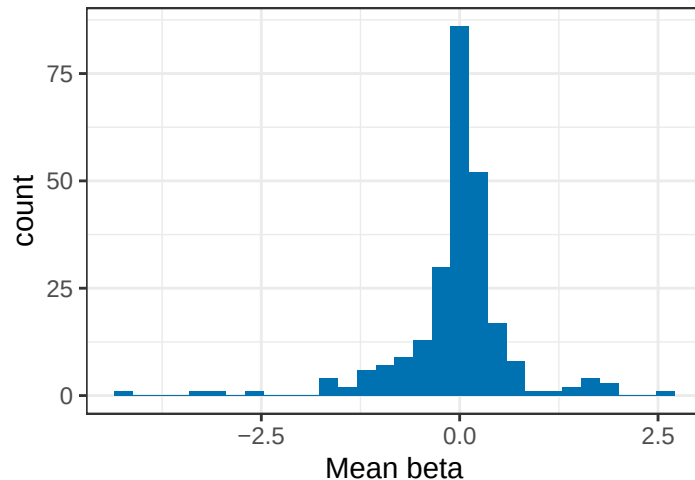
```

Prior predictive checks

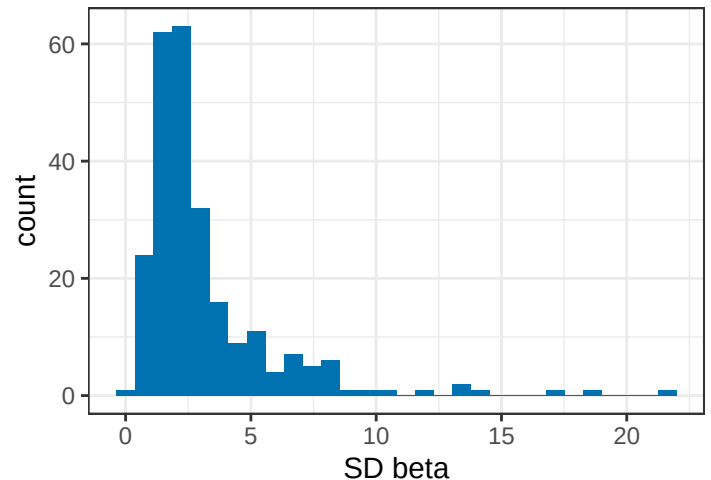
A Distribution of simulated betas



B Means of simulated betas



C SDs of simulated betas



This all looks good and we go ahead with these priors and check the results of the SBC. We only plot the results from the models that had no divergence issues.

```

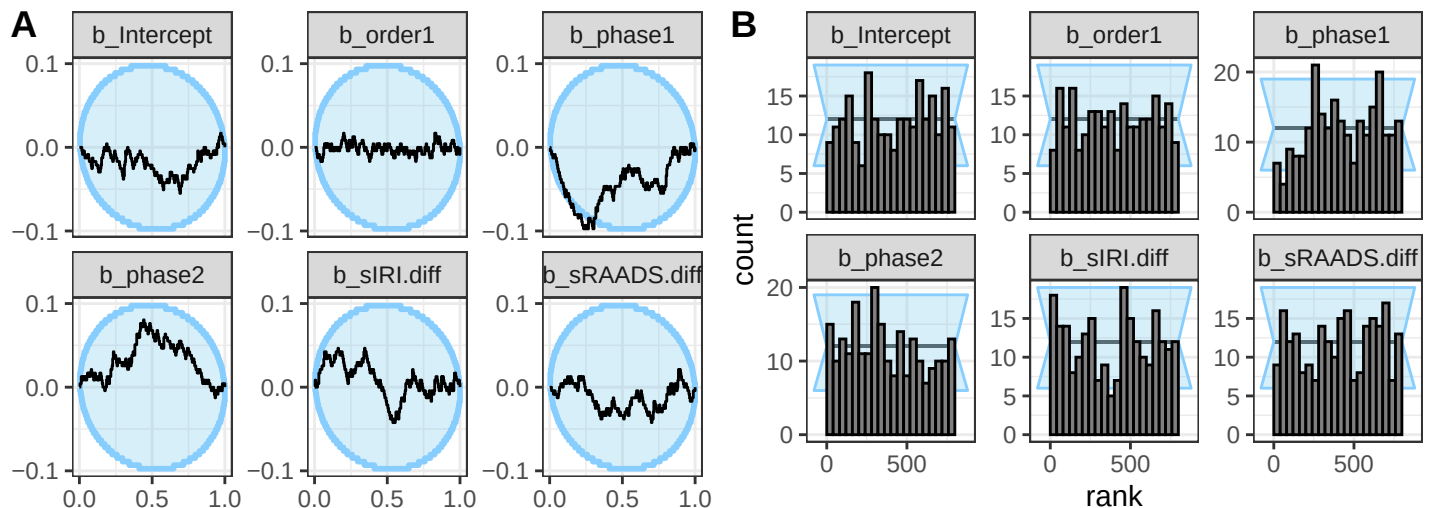
# get simulation numbers with issues
mx_rnk = max(df.results$max_rank)
check = merge(df.results %>%
  group_by(sim_id) %>%
  summarise(rhat = max(rhat, na.rm = T),
    max_rank = mean(max_rank)) %>%
  filter(rhat >= 1.05 | max_rank != mx_rnk),
  df.backend %>% filter(n_divergent > 0), all = T)

# plot SBC with functions from the SBC package focusing on population-level parameters
df.results.b = df.results %>%
  filter(substr(variable, 1, 2) == "b_") %>%
  filter(!(sim_id %in% check$sim_id))
p1 = plot_ecdf_diff(df.results.b) + theme_bw() + theme(legend.position = "none") +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p2 = plot_rank_hist(df.results.b, bins = 20) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))

p = ggarrange(p1, p2, labels = "AUTO", ncol = 2, nrow = 1)
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
  face = "bold", size = 14))

```

Computational faithfulness and model sensitivity



All looks good and we judge this to be acceptable as we cannot identify clear bias patterns as described in the information accompanying the SBC package: https://hyunjimoon.github.io/SBC/articles/rank_visualizations.html

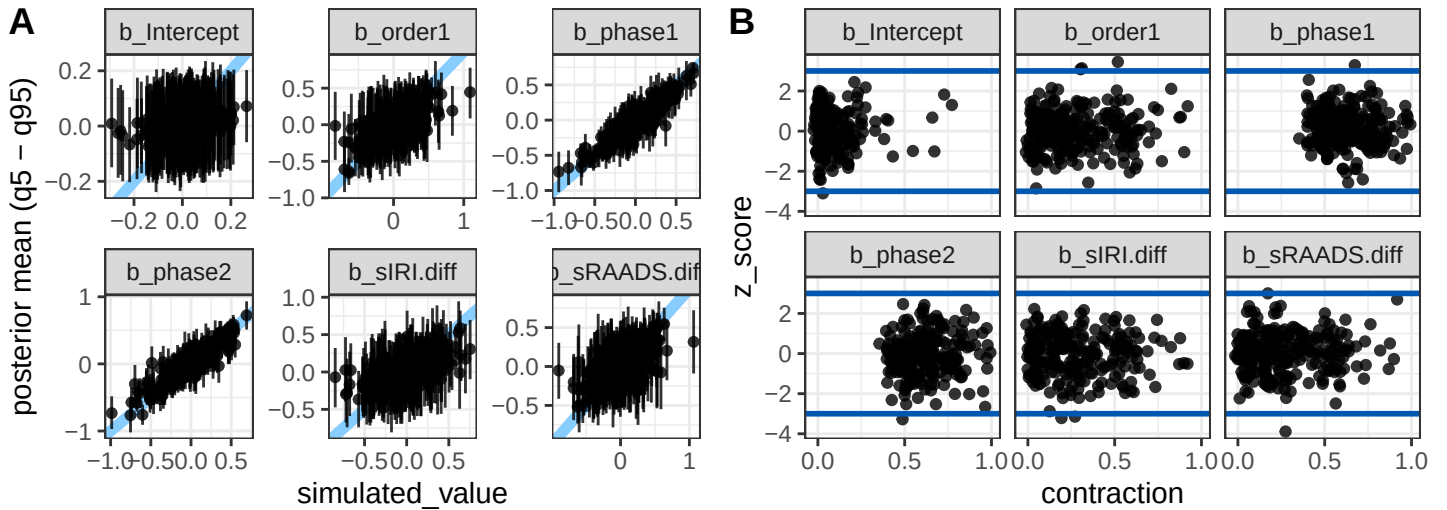
```

p3 = plot_sim_estimated(df.results.b, alpha = .8) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p4 = plot_contraction(df.results.b,
  prior_sd =
    setNames(c(0.1, rep(0.3, times = length(unique(df.results.b$variable))-1)),
      unique(df.results.b$variable))) +
  theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3)) +
  geom_hline(yintercept = c(-3, 3), colour = c_dark_highlight, linewidth = 1)

p = ggarrange(p3, p4, labels = "AUTO", ncol = 2, nrow = 1)
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
  face = "bold", size = 14))

```

Computational faithfulness and model sensitivity



Then, we investigated the relationship between the simulated true parameters and the posterior estimates. Most parameters were recovered successfully within an uncertainty interval of $\alpha = 0.05$. Last, z-score and the posterior contraction of our population-level predictors look good as well.

3.2 Level of the individual

```
# create the correct dataframe
df.indi.mg.mea = df.indi.mg.mea %>%
  merge(., df.indi %>% select(subID, role, RAADS_final, IRI_final)) %>%
  mutate(
    # recode the phase to capture whether someone was leading or not
    phase.orig = phase,
    phase = case_when(
      phase == "p1" & role == "Leader1" ~ "pL",
      phase == "p2" & role == "Leader1" ~ "pF",
      phase == "p1" & role == "Leader2" ~ "pF",
      phase == "p2" & role == "Leader2" ~ "pL",
      phase == "p3" ~ "pB"
    )
  ) %>%
  droplevels() %>% ungroup() %>%
  mutate(across(where(is.numeric),
    .fns = scale_this,
    .names = "s{.col}")) %>%
  mutate_if(is.character, as.factor)

# set the contrasts
contrasts(df.indi.mg.mea$phase) = contr.sum(3)[c(3,1,2),]
contrasts(df.indi.mg.mea$phase)

##      [,1] [,2]
## pB    -1   -1
## pF     1    0
## pL     0    1

contrasts(df.indi.mg.mea$order) = contr.sum(2)
contrasts(df.indi.mg.mea$order)

##      [,1]
## CT-MG    1
## MG-CT   -1
```

```

# code for filenames
code = "MG_indi_MEA_mean"

# model formula
f = brms::bf( sMEA.mean ~ sRAADS_final + sIRI_final + phase + order +
              (1 | dyad))
f

## sMEA.mean ~ sRAADS_final + sIRI_final + phase + order + (1 | dyad)

# check if the SBC already exists
if (file.exists(file.path(cache_dir, sprintf("df_res_%s.rds", code)))) {
  # load in the results of the SBC
  df.results = readRDS(file.path(cache_dir, sprintf("df_res_%s.rds", code)))
  df.backend = readRDS(file.path(cache_dir, sprintf("df_div_%s.rds", code)))
  dat        = readRDS(file.path(cache_dir, sprintf("dat_%s.rds", code)))
} else {
  # stimulate some data
  set.seed(2848)
  gen = SBC_generator_brms(f, data = df.indi.mg.mea,
                          prior = priors,
                          thin = 50, warmup = 10000, refresh = 2000,
                          generate_lp = TRUE, init = 0.1)
  bck = SBC_backend_brms_from_generator(gen, chains = 4, thin = 1,
                                       warmup = warm, iter = iter)
  dat = generate_datasets(gen, nsim)
  saveRDS(dat, file.path(cache_dir, sprintf("dat_%s.rds", code)))
  # perform the SBC
  res = compute_SBC(
    dat,
    bck,
    cache_mode      = "results",
    cache_location = file.path(cache_dir, sprintf("res_%s", code)))
  df.results = res$stats
  df.backend = res$backend_diagnostics
  saveRDS(df.results, file = file.path(cache_dir, paste0("df_res_", code, ".rds")))
  saveRDS(df.backend,
          file = file.path(cache_dir, paste0("df_div_", code, ".rds")))
}

```

We start by investigating the rhats and the number of divergent samples. This shows that only 0 of 250 simulations had at least one parameter that had an rhat of at least 1.05 and 7 models had divergent samples. This suggests that this model performs well, but again we will check for divergence issues in the final model. .

Next, we can plot the simulated values to perform prior predictive checks.

```

dvfakemat = matrix(NA, nrow(dat[['generated']][[1]]), length(dat[['generated']]))
for (i in 1:length(dat[['generated']])) {
  dvfakemat[,i] = dat[['generated']][[i]][[1]]
}

# compute one histogram per simulated data-set
dvfakematH = dvfakemat
dvfakematH[dvfakematH > 4] = 4
dvfakematH[dvfakematH < -4] = -4
breaks = seq(min(dvfakematH, na.rm=T), max(dvfakematH, na.rm=T), length.out = 101)
binwidth = breaks[2] - breaks[1]
histmat = matrix(NA, ncol = dim(dvfakemat)[2], nrow = length(breaks)-1)
for (i in 1:dim(dvfakemat)[2]) {
  histmat[,i] = hist(dvfakematH[,i], breaks = breaks, plot = F)$counts
}

# for each bin, compute quantiles across histograms

```

```

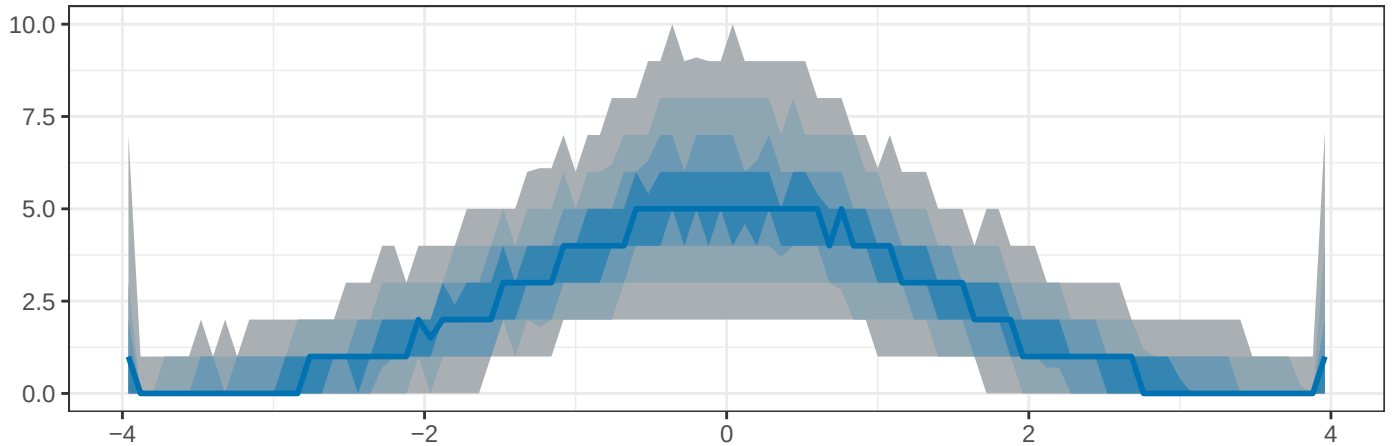
probs = seq(0.1, 0.9, 0.1)
quantmat = as.data.frame(matrix(NA, nrow=dim(histmat)[1], ncol = length(probs)))
names(quantmat) = paste0("p", probs)
for (i in 1:dim(histmat)[1]) {
  quantmat[i,] = quantile(histmat[i,], p = probs)
}
quantmat$x = breaks[2:length(breaks)] - binwidth/2 # add bin mean
p1 = ggplot(data = quantmat, aes(x = x)) +
  geom_ribbon(aes(ymax = p0.9, ymin = p0.1), fill = c_light) +
  geom_ribbon(aes(ymax = p0.8, ymin = p0.2), fill = c_light_highlight) +
  geom_ribbon(aes(ymax = p0.7, ymin = p0.3), fill = c_mid) +
  geom_ribbon(aes(ymax = p0.6, ymin = p0.4), fill = c_mid_highlight) +
  geom_line(aes(y = p0.5), colour = c_dark, linewidth = 1) +
  labs(title = "Distribution of simulated betas", y = "", x = "") +
  theme_bw()

tmpM = apply(dvfakemat, 2, mean) # mean
tmpSD = apply(dvfakemat, 2, sd)
p2 = ggplot() +
  stat_bin(aes(x = tmpM), fill = c_dark) +
  labs(x = "Mean beta", title = "Means of simulated betas") +
  theme_bw()
p3 = ggplot() +
  stat_bin(aes(x = tmpSD), fill = c_dark) +
  labs(x = "SD beta", title = "SDs of simulated betas") +
  theme_bw()
p = ggarrange(p1,
  ggarrange(p2, p3, ncol = 2, labels = c("B", "C")),
  nrow = 2, labels = "A")
annotate_figure(p, top = text_grob("Prior predictive checks", face = "bold", size = 14))

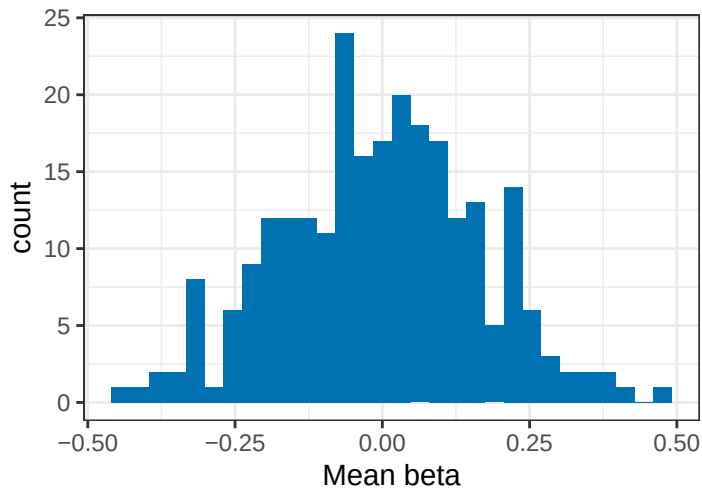
```

Prior predictive checks

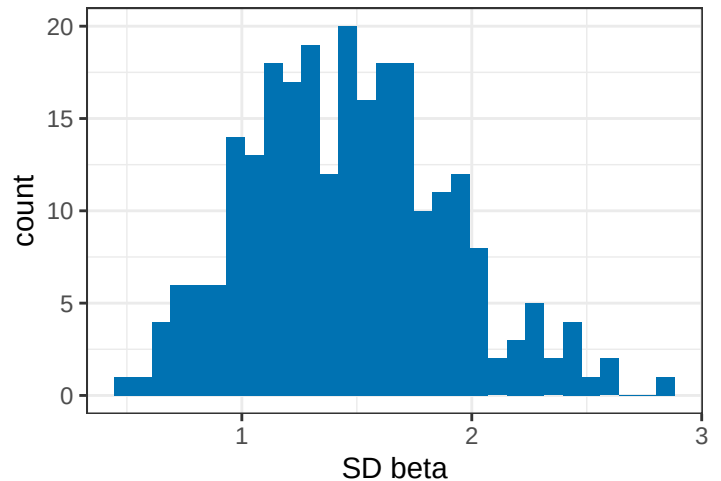
A Distribution of simulated betas



B Means of simulated betas



C SDs of simulated betas



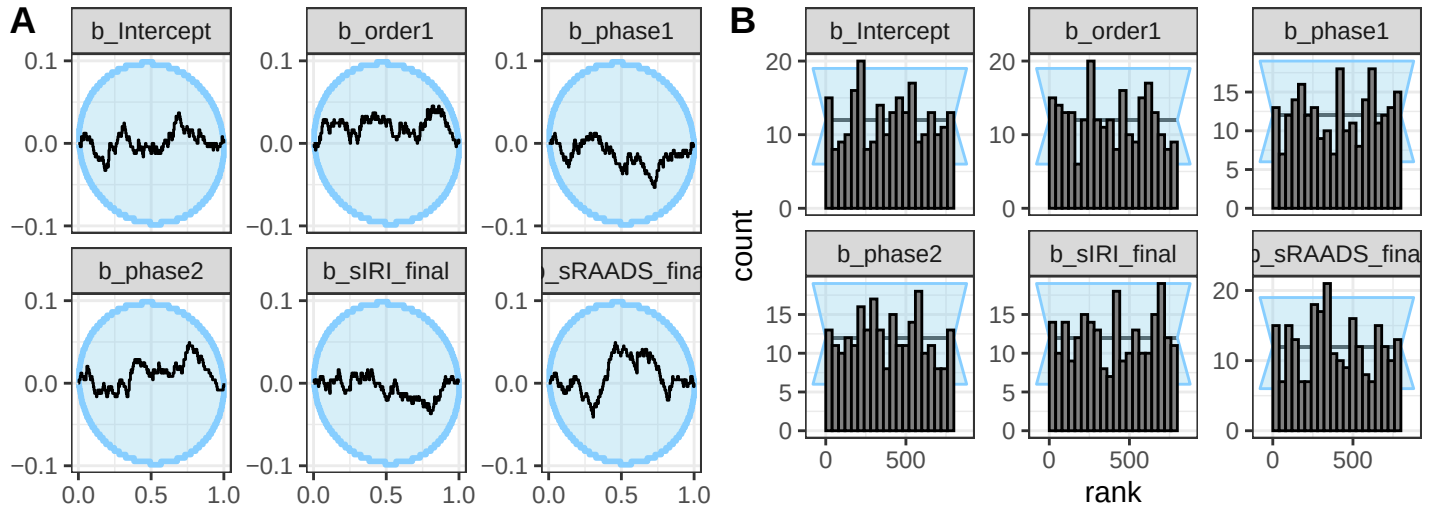
This all looks good and we go ahead with these priors and check the results of the SBC. We only plot the results from the models that had no divergence issues.

```
# get simulation numbers with issues
mx_rnk = max(df.results$max_rank)
check = merge(df.results %>%
  group_by(sim_id) %>%
  summarise(rhat = max(rhat, na.rm = T),
            max_rank = mean(max_rank)) %>%
  filter(rhat >= 1.05 | max_rank != mx_rnk),
  df.backend %>% filter(n_divergent > 0), all = T)

# plot SBC with functions from the SBC package focusing on population-level parameters
df.results.b = df.results %>%
  filter(substr(variable, 1, 2) == "b_") %>%
  filter(!(sim_id %in% check$sim_id))
p1 = plot_ecdf_diff(df.results.b) + theme_bw() + theme(legend.position = "none") +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p2 = plot_rank_hist(df.results.b, bins = 20) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p = ggarrange(p1, p2, labels = "AUTO", ncol = 2, nrow = 1)
```

```
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
                                   face = "bold", size = 14))
```

Computational faithfulness and model sensitivity

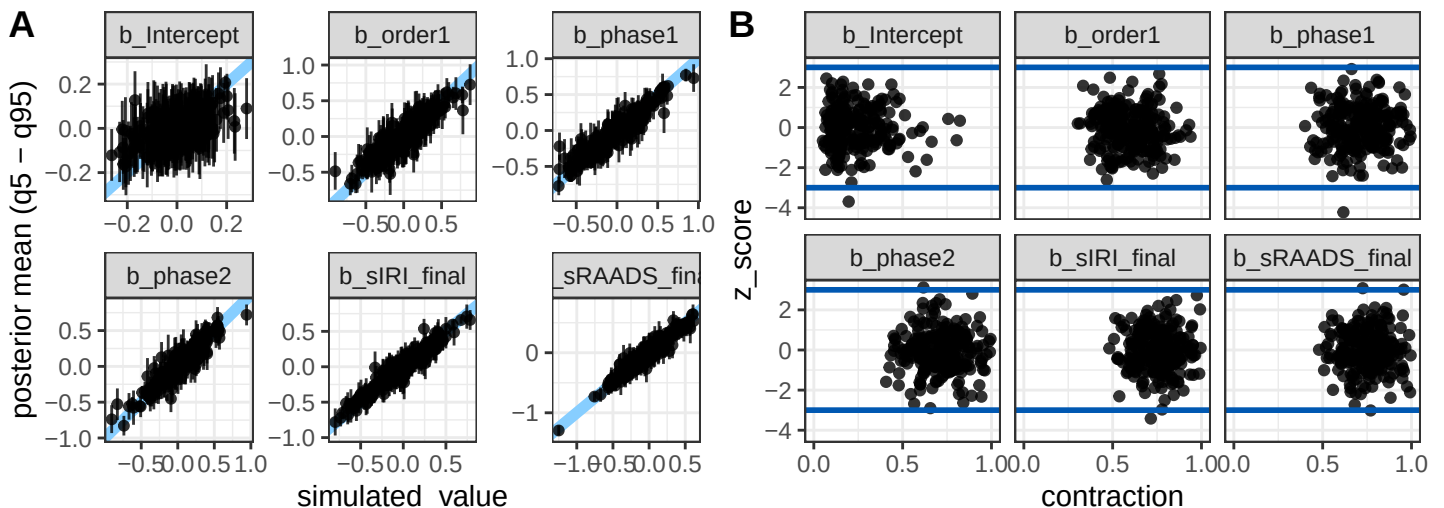


All looks good and we judge this to be acceptable as we cannot identify clear bias patterns as described in the information accompanying the SBC package: https://hyunjimoon.github.io/SBC/articles/rank_visualizations.html

```
p3 = plot_sim_estimated(df.results.b, alpha = .8) + theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3))
p4 = plot_contraction(df.results.b,
  prior_sd =
    setNames(c(0.1, rep(0.3, times = length(unique(df.results.b$variable))-1)),
      unique(df.results.b$variable))) +
  theme_bw() +
  scale_x_continuous(breaks=scales::pretty_breaks(n = 3)) +
  scale_y_continuous(breaks=scales::pretty_breaks(n = 3)) +
  geom_hline(yintercept = c(-3, 3), colour = c_dark_highlight, linewidth = 1)

p = ggarrange(p3, p4, labels = "AUTO", ncol = 2, nrow = 1)
annotate_figure(p, top = text_grob("Computational faithfulness and model sensitivity",
                                   face = "bold", size = 14))
```

Computational faithfulness and model sensitivity



Then, we investigated the relationship between the simulated true parameters and the posterior estimates. Most parameters were recovered successfully within an uncertainty interval of $\alpha = 0.05$. Last, z-score and the posterior contraction of our population-level predictors look good as well.